

Lab 5 Exam

文件系统服务的添加流程还是非常复杂的，虽然 exam 中已经给出了全部流程，将题目中给的代码复制一下即可，但 extra 就不会给出太详细的流程了，建议大家还是对照着题目中“实现思路”一节好好捋顺一下整个流程，对以后写挑战性任务也很有帮助。

将题目中的代码复制完成之后，我们需要完成的就只剩下 fs/fs.c 中的 find_files() 和 traverse_file() 两个函数了。

首先我们来完成 find_files() 函数，需要调用 walk_path() 函数找到路径名 path 对应的文件，这就需要我们充分了解 walk_path() 函数的构成。

walk_path() 函数定义为 int walk_path(char *path, struct File **pdir, struct File **pfile, char *lastelem)，其中 char *path 为要寻找的文件的路径，其余三个参数用来存储返回值。若成功找到文件，则 struct File **pdir 为该文件所在的文件夹，struct File **pfile 为该文件本身；若未成功找到文件，但找到了该文件所在的文件夹，则 struct File **pdir 为该文件所在的文件夹，char *lastelem 为该文件的名称。除成功找到文件函数返回 0 以外，其它未找到文件的情况函数均返回负值错误码。

我们现在可以发现，在 find_files() 函数中调用 walk_path() 函数时，是完全用不到第二和第四个参数的，只需要保证在 path 对应的文件存在的情况下能够将文件本身取出就可以了。当然，为了防止访问空指针的情况，我们还是在这些参数的位置塞一些东西进去为好，虽然我们拿到这些数据以后也会转手把它们扔掉。

最后按照题目中定义调用 traverse_file() 函数即可：

```
// fs/fs.c
int find_files(const char *path, const char *name, struct File *file = NULL;
// Lab5-Exam: Your code here. (1/2)
struct File *pdir = NULL; // actually no use
char nname[MAXNAMELEN]; // actually no use
int r = walk_path(path, &pdir, &file, nname); // 只
if (r < 0) {
    return r;
}
// Lab5-Exam: Your code here. (2/2)
traverse_file(path, file, name, res); // 按定义调用
return 0;
}
```

接下来我们来完成 traverse_file() 函数，根据注释一步一步完成：

1. 检查路径长度是否符合要求，如不符合，直接返回

根据注释提示，若 path 的长度为零或不小于最大路径长度，则返回 -E_BAD_PATH 错误码，使用 strlen() 函数即可：

```
// fs/fs.c
if (strlen(path) == 0 || strlen(path) >= MAXPATHLEN /*
    /*返回*/
    return -E_BAD_PATH;
}
```

2. 比较当前文件名是否等于 `name`，如果相等则更改 `res`

观察 `res` 的结构，我们知道 `int` 型变量 `res->count` 存储着查找到的文件的总数量，而字符串数组 `res->file_path[]` 存储着每个查找到的文件的路径。所以只需要更新这两个变量即可：

```
// fs/fs.c
if (!strcmp(file->f_name, name) /*file 的名字等于 name*/
    /*增加 res->count*/
    /*添加 res 的路径*/
    strcpy(res->file_path[res->count], path);
    res->count++;
}
```

3. 把 `path` 和 `name` 拼接起来得到下一层文件路径，注意结尾的 `'\0'`
提示：我们没有实现 `strcat` 工具函数，你可以用 `strcpy` 实现拼接

这一步看似十分简单，实则却暗藏玄机。就因为这一步，我在考场上被硬控了 70 分钟，甚至有好多同学直接被硬控两个半小时（是的，这次考试延时了半小时，也是这一届唯一一次 OS 上机延时），最后连 exam 都没做上。

本质原因其实是：我们在拼接的时候并不知道 `path` 的结尾到底有没有 `/` ！

你或许会问，题目中不是已经保证了 `path` 均以 `/` 结尾吗？实际上，`traverse_file()` 是一个递归函数，这里 `path` 的值不仅可能是最初输入的 `path`，还可能是其中任意一个文件夹的路径，所以无法保证 `path` 的结尾一定有 `/`。

为此，在拼接的时候，我们需要检查当前 `path` 的结尾是不是 `/`。如果不是的话，就需要在当前 `path` 的结尾补一个 `/`：

```
// fs/fs.c
strcpy(curpath, path);
int curlen = strlen(curpath);
// 如果 path 不以 '/' 结尾，在后面加一个 '/'
if (curpath[curlen - 1] != '/') {
    curpath[curlen] = '/';
    curlen++; // 别忘了 len++
}
strcpy(curpath + curlen, f->f_name);
```

```
int flen = strlen(f->f_name);
curpath[curlen + flen] = '\\0'; // 别忘了最后加 '\\0'
```

4. 递归调用 `traverse_file` 函数

最后按照定义递归调用 `traverse_file()` 函数，这道题就全部做完了：

```
// fs/fs.c
traverse_file(curpath, f, name, res);
```

还是总结一下吧，这场 exam 对我来说实在是一场苦战，导致最后也没有太多时间做 extra 了，即使续费了半个小时也还是没写完，实在是太痛苦了。考场上确实很容易钻牛角尖，在高压状况下大脑很容易宕机，还是祝愿各位能够保持一个稳定平和的心态，以冷静的头脑和灵活的思维面对层出不穷的突发状况。

Lab 5 Extra

可怜的贾爹因为这道题让人唠一辈子，其实这个题或许真的没有大家想象中那么难，除了题量大了点以外真没啥毛病，只要是认真研读了 Lab 5 的代码，都大有做上的希望。

题目就不带大家重新读了，简单来说，除了题目中已经完全给出的代码，我们还需要完成 `fs/serv.c` 中的 `serve_key_set()` `serve_key_unset()` `serve_key_isset()` 3 个函数，`user/lib/fsipc.c` 中的 `fsipc_key_set()` `fsipc_key_unset()` `fsipc_key_isset()` 3 个函数，`user/lib/file.c` 中的 `fskey_set()` `fskey_unset()` `fskey_isset()` 3 个函数，以及修改 `fs/serv.c` 中的 `serve_map()` `serve_close()` 2 个函数。不过不要太害怕，在这 11 个函数中，有 5 个函数都只需要一行就可以解决，当然这属于是未卜先知了，我们下面正式开始解题：

首先是题目中的第 4 步，我们需要根据注释中的提示，完成 `fs/serv.c` 中的 3 个 `serv` 函数。`fs/serv.c` 中有两个全局变量，分别是 `int` 类型的 `encrypt_key_set` 和 `unsigned char[BLOCK_SIZE]` 类型的 `encrypt_key`。`encrypt_key_set` 用于判断密钥是否加载到了缓存中，或者说当前密钥是否可用。根据题目中的定义，当密钥加载到缓存中时，我们需要返回 1，否则返回 0，于是我们在两种情况下将 `encrypt_key_set` 分别置为 1 或 0 即可。`encrypt_key` 记录着密钥的内容，大小为一个磁盘块。对于每个需要加密或解密的磁盘块，我们都将其与 `encrypt_key` 按位异或，即可完成加密或解密。

我们首先来看 `serve_key_set()` 函数。我们首先需要判断密钥是否已加载，只需要判断 `encrypt_key_set` 是否为 1 即可。这里需要注意的是返回值的方式，我们不能直接使用 `return` 语句，因为我们要将返回值传递给用户进程，所以需要使用 `ipc_send()` 函数来返回。`ipc_send()` 函数可以返回一个值和一个页面，在整个这道题中，我们都不涉及到返回页面，只需要返回一个值即可，此时我们使用 `ipc_send(envid, -E_BAD_KEY, 0, 0)`，即可将 `-E_BAD_KEY` 返回到 `envid` 对应的进程中，`ipc_send()` 后面的两个参数置 0 即可。在使用 `ipc_send()` 函数返回之后，切记要使用 `return;` 直接返回，否则会继续往下执行。

接下来我们使用 `open_lookup()` 函数找到对应的 `Open` 结构体，既然已经给出要使用的函数了，那就比较简单了。我们直接向 `open_lookup()` 函数输入 `envid` 和 `fileid`，即可得到与 `fileid` 对应的 `Open` 结构体。或许你会有疑问，`fileid` 何时来的，实际上，只需要看一下参数中的请求 `rq` 的结构，就可以发现这个结构体中唯一包含的就是 `rq_fileid` 字段，代表着密钥文件的 `fileid`。

接下来，我们需要对刚才得到的密钥文件的 `Open` 结构体进行检查，检查文件大小是否大于 2 个磁盘块，以及第一个磁盘块中的第一个字是否为 `FS_MAGIC`。首先是文件大小，`Open` 结构体中有一个 `struct File *` 类型的 `o_file` 字段，指向该文件的文件控制块；文件控制块里又有一个 `uint32_t` 类型的 `f_size` 字段，表示该文件的大小。接下来是文件的第一个磁盘块，我们根据注释中的提示，使用 `fs/fs.c` 中

的 `file_get_block()` 函数，输入文件的文件控制块 `f` 和磁盘块序号 `filebno`（这里注意第一个磁盘块的序号是 0），即可将对应的磁盘块读出到块缓存中，返回的 `*blk` 即为该磁盘块的虚拟地址。在得到了第一个磁盘块的虚拟地址之后，我们只需要将 `void *` 类型的 `blk` 强制转换为 `u_int *` 类型（其实 `int *` 类型也可以，无所谓），再使用 `blk[0]` 即可读取该地址下的第一个字，与 `FS_MAGIC` 这个魔数进行比较即可。

接下来我们如法炮制，读取加密文件中的第二个磁盘块（注意序号是 1），将该磁盘块中的内容复制到 `encrypt_key` 中，这里可以使用 `for` 循环，也可以使用 `memcpy()` 函数。最后，我们将当前状态标记为已加载密钥，即将 `encrypt_key_set` 置为 1，然后使用 `ipc_send(envid, 0, 0, 0)` 返回 0 即可：

```
// fs/serv.c
void serve_key_set(u_int envid, struct Fsreq_key_set *r)
{
    // 判断当前状态是否已加载密钥，如果已加载密钥，IPC 返回 0
    if (encrypt_key_set) {
        ipc_send(envid, -E_BAD_KEY, 0, 0);
        return;
    }

    // 利用 open_lookup 找到对应的 Open 结构体，判断文件大小
    int r;
    struct Open *pOpen;
    if ((r = open_lookup(envid, rq->req_fileid, &pOpen)) < 0)
        ipc_send(envid, r, 0, 0);
    return;
}

if (pOpen->o_file->f_size < 2 * BLOCK_SIZE) {
    ipc_send(envid, -E_INVALID_KEY_FILE, 0, 0);
    return;
}

// 利用 file_get_block 读取文件的第一个磁盘块，判断第一个字是否是魔数
void *blk;
if ((r = file_get_block(pOpen->o_file, 0, &blk)) < 0)
    ipc_send(envid, r, 0, 0);
return;
}

// 如果密钥文件不合法，IPC 返回 -E_INVALID_KEY_FILE
if (((int *)blk)[0] != FS_MAGIC) {
    ipc_send(envid, -E_INVALID_KEY_FILE, 0, 0);
    return;
}

// 利用 file_get_block 读取文件的第二个磁盘块，将密钥复制到 encrypt_key 中
void *anoblk;
if ((r = file_get_block(pOpen->o_file, 1, &anoblk)) < 0)
    ipc_send(envid, r, 0, 0);
return;
}
```

```

        return;
    }
    memcpy(encrypt_key, (unsigned char *)anoblk, BLOCK_

// 将当前状态标记为已加载密钥
encrypt_key_set = 1;

// IPC 返回 0
ipc_send(envid, 0, 0, 0);
}

```

接下来是 `serve_key_unset()` 函数。首先判断当前是否已加载密钥，并将当前状态标记为未加载密钥，这一步仿照刚才去做即可。接下来将 `encrypt_key` 清零，这里依然可以使用 `for` 循环，当然也可以使用 `memset()` 函数实现。最后我们使用 `ipc_send()` 返回 0 即可：

```

// fs/serv.c
void serve_key_unset(u_int envid) {
    // 判断当前状态是否已加载密钥，如果未加载密钥，IPC 返回
    if (!encrypt_key_set) {
        ipc_send(envid, -E_BAD_KEY, 0, 0);
        return;
    }

    // 将当前状态标记为未加载密钥
    encrypt_key_set = 0;
}

```

然后是 `serve_key_isset()` 函数。这个函数只需要直接返回当前密钥的加载状态，即 `encrypt_key_set` 即可：

```

// fs/serv.c
void serve_key_isset(u_int envid) {
    // IPC 返回当前状态
    ipc_send(envid, encrypt_key_set, 0, 0);
}

```

接下来是第 6 步和第 7 步，我们继续在 `fs/serv.c` 中修改 `serve_map()` 函数和 `serve_close()` 函数。

在 `serve_map()` 函数中，当打开文件的 `mode` 中含有 `O_ENCRYPT` 时，我们需要检查当前是否加载了密钥，如果加载了密钥，我们就需要对当前的磁盘块进行加密。首先，我们需要获取当前打开文件的 `mode`，在 `Open` 结构体 `pOpen` 中有一个 `o_mode` 字段，我们只要将其与 `O_ENCRYPT` 进行按位与运算，即可判断出该文件是否以加密形式打开。

如果以加密形式打开，我们就需要将当前磁盘块的内容 `blk` 与密钥内容 `encrypt_key` 按位异或，这样就完成了 `serve_map()` 函数：

```
// fs/serv.c
void serve_map(u_int envid, struct Fsreq_map *rq) {
    struct Open *pOpen;
    u_int filebno;
    void *blk;
    int r;

    if ((r = open_lookup(envid, rq->req_fileid, &pOpen))
        ipc_send(envid, r, 0, 0);
        return;
    }

    filebno = rq->req_offset / BLOCK_SIZE;

    if ((r = file_get_block(pOpen->o_file, filebno, &blk))
        ipc_send(envid, r, 0, 0);
        return;
    }

    if (pOpen->o_mode & O_ENCRYPT) {
        if (!encrypt_key_set) {
            ipc_send(envid, -E_BAD_KEY, 0, 0);
            return;
        }
        else {
            for (int i = 0; i < BLOCK_SIZE; i++) {
                *((unsigned char *) (blk + i)) = (*((unsigned char *) (blk + i)) ^ encrypt_key[i]);
            }
        }
    }

    ipc_send(envid, 0, blk, PTE_D | PTE_LIBRARY);
}
```

`serve_close()` 函数的修改略微有些难度，当打开文件的 `mode` 中含有 `O_ENCRYPT` 时，我们需要检查当前是否加载了密钥，如果加载了密钥，我们就需要对文件中的所有磁盘块进行解密。和之前不同的是，我们需要加密或解密的不只是一个磁盘块，而是文件中的所有磁盘块，这就需要我们根据 `Open` 结构体 `pOpen` 依次找到所有的磁盘块 `blk`。根据提示，我们可以参考 `fs/fs.c` 中的 `file_flush()` 函数，通过文件大小计算文件的磁盘块数，根据文件的磁盘块数遍历磁盘块序号，使用 `fs/fs.c` 中的 `file_get_block()` 函数即可遍历到所有的磁盘块。

刚才已经介绍过了，我们可以通过 `pOpen->o_file->f_size` 获取到文件的大小。接下来参考 `file_flush()` 函数中的 `u_int nblocks = ROUND(f->f_size, BLOCK_SIZE) / BLOCK_SIZE`；即可得到文件的磁盘块数 `nblocks`。将磁盘块序号从 0 遍历到 `nblocks - 1`，将文件控制块 `pOpen->o_file` 和磁盘块序号输入 `file_get_block()` 函数中，即可得到当前磁盘块序号对应的磁盘块：

```
// fs/serv.c
void serve_close(u_int envid, struct Fsreq_close *rq) {
    struct Open *pOpen;

    int r;

    if ((r = open_lookup(envid, rq->req_fileid, &pOpen))
        ipc_send(envid, r, 0, 0);
        return;
    }

    if (pOpen->o_mode & O_ENCRYPT) {
        if (!encrypt_key_set) {
            ipc_send(envid, -E_BAD_KEY, 0, 0);
            return;
        }
        else {
            struct File *f = pOpen->o_file;
            u_int nblocks = ROUND(f->f_size, BLOCK_SIZE) / BLOCK_SIZE;
            for (u_int bno = 0; bno < nblocks; bno++) {
                void *blk;
                if ((r = file_get_block(f, bno, &blk))
                    continue;
                }
                for (int i = 0; i < BLOCK_SIZE; i++) {
                    *((unsigned char *)(blk + i)) = (*(
                }
            }
        }
    }

    file_close(pOpen->o_file);
    ipc_send(envid, 0, 0, 0);
}
```

最后我们来完成第 8 步，完成 `user/lib/fsipc.c` 和 `user/lib/file.c` 中的 6 个函数。我们首先来看 `user/lib/fsipc.c` 中的 3 个函数，这些函数的实现方式非常固定，主要作用就是将参数打包进 `req` 结构体中，作为传递给文件进程 `serv` 服务的参数，并且在 `return` 语句中调用 `fsipc()` 函数。

对于 `fsipc_key_set()` 函数，我们仿照同文件中的其它有参数的函数，例如 `fsipc_close()` `fsipc_dirty()` `fsipc_remove()`，先将 `fsipcbuf` 强制转换为对应的 `req` 结构体，即 `struct Fsreq_key_set *` 类型，再将参数中的 `fileid` 写入 `req` 结构体中即可。最后，我们在 `return` 语句中调用 `fsipc()` 函数，第一个参数是函数对应的服务，第二个参数为 `req`。由于我们只需要通过 `ipc` 从文件进程接收一个值，而不需要接收页面，所以将 `fsipc()` 函数的第三个和第四个参数置 0 即可。

对于 `fsipc_key_unset()` 和 `fsipc_key_isset()` 两个函数，它们没有参数需要传递给文件进程，所以实际上不需要 `req` 参数，我们仿照同文件中的 `fsipc_sync()` 函数，在调用 `fsipc()` 函数时在第二个参数的位置直接填入 `fsipcbuf` 即可，实际上这个 `fsipcbuf` 并不会被文件进程使用到：

```
// user/lib/fsipc.c
int fsipc_key_set(u_int fileid) {
    // 合理调用 fsipc 函数
    struct Fsreq_key_set *req = (struct Fsreq_key_set *)
    req->req_fileid = fileid;
    return fsipc(FSREQ_KEY_SET, req, 0, 0);
}

int fsipc_key_unset(void) {
    // 合理调用 fsipc 函数
    return fsipc(FSREQ_KEY_UNSET, fsipcbuf, 0, 0);
}

int fsipc_key_isset(void) {
    // 合理调用 fsipc 函数
    return fsipc(FSREQ_KEY_ISSET, fsipcbuf, 0, 0);
}
```

最后，我们再来看 `user/lib/file.c` 中的 3 个函数。首先是 `fskey_set()` 函数。根据提示，我们需要先使用 `fd_lookup()` 函数找到密钥文件的文件描述符，我们向 `fd_lookup()` 函数输入函数的参数 `fdnum`，即可得到文件描述符 `fd`。

接下来我们需要检查密钥文件的打开模式，文件描述符 `fd` 中同样有一个字段 `fd_omode`，记录着当前文件的打开模式。首先密钥文件不能够以加密模式打开，于是我们可以通过 `fd->fd_omode` 与 `O_ENCRYPT` 按位异或来判断。另外，密钥文件必须通过允许读的模式打开，这里需要注意的是，允许读写模式的判断方式比较特殊，这里有一个比较奇怪的设计：在宏定义中，只读模式 `O_RDONLY` 为 `0x0000`，只写模式 `O_WRONLY` 为 `0x0001`，读写模式 `O_RDWR` 为 `0x0002`，所以通过直接按位与的方式来判断模式的方法在这里是错误的，例如 `fd->fd_omode` 与 `0x0000` 按位与的结果显然是恒为 0；正确的做法是将 `fd->fd_omode` 与 `O_ACCMODE`（即 `0x0003`）进行按位与，判断按位与的结果分别与 `O_RDONLY` `O_WRONLY` `O_RDWR` 是否相等，具体做法可以参考 `user/lib/fd.c` 中的 `read()` 函数和 `write()` 函数，注意双等于号 `==` 的优先级高于按位与 `&`，所以必须为按位与表达式加上括号！回到题目，使用 `O_RDONLY` 模式或 `O_RDWR` 模式打开密钥文件都是可以的，只有使用 `O_WRONLY` 模式打开密钥文件是错误的。

在 `return` 语句中，我们需要调用相应的 IPC 函数，即 `fsipc_key_set()` 函数。这个函数需要一个名为 `fileid` 的参数。然而我们能够发现，`f_fileid` 字段并不位于 `Fd` 结构体中，而是位于 `Filefd` 中。不过没关系，我们知道这两种结构体可以互相转换，我们直接将当前的文件描述符 `fd` 强制转换为 `struct Filefd *` 类型，即可获得其中的 `f_fileid` 字段了：


```
// user/lib/file.c
int fskey_set(int fdnum) {
    // 使用 fd_lookup 找到对应的 Fd 结构体。判断传入的文件
    // 如果不合法返回 fd_lookup 函数的返回值（该函数除了0之外
    int r;
    struct Fd *fd;
    if ((r = fd_lookup(fdnum, &fd)) < 0) {
        return r;
    }
}
```

最后，我们完成 fskey_unset() 和 fskey_isset() 两个函数，只需要直接调用相关的 IPC 函数，也就是 fsipc_key_unset() 和 fsipc_key_isset() 就可以了：

```
// user/lib/file.c
int fskey_unset() {
    // 合理调用相应文件系统 IPC 函数
    return fsipc_key_unset();
}

int fskey_isset() {
    // 合理调用相应文件系统 IPC 函数
    return fsipc_key_isset();
}
```

这样，我们就完成这道史诗级的 Lab 5 extra！其实这道题的难度并没有拉满，甚至完全没有出现需要传递页面的 IPC，不过有很多细节还是我们需要对课下代码比较全面的理解才能够写对，比如之前提到的读写模式的检测，所以实际上能够做出来的同学依然是寥寥无几。这也提醒我们课下如果不认真读代码，不做好全面的准备，甚至完全不读代码，直接抄开源代码，想要在考试中答对所有问题肯定是不可能的（当然如果你本来就没打算做 extra 那当我没说哈哈）。文件系统这一章的内容非常之多，函数调用错综复杂，还是希望大家能够仔细研读，加油！